

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Zeiger

- Parameter des Hauptprogramms

- Zeiger und `const`

- Zeigerarithmetik

- Operationen mit Zeigern

- Zeiger als Parameter (II)

- Zeigervektoren

- Zeiger auf Zeiger

Nullzeiger

Es gibt einen Makro `NULL`, der den **Nullzeiger** definiert.

Der Wert dieses Makros muss von den Implementatoren so gewählt werden, dass kein *echter* Zeiger ihn haben kann.

Der Nullzeiger erlaubt die Markierung von Zeigervariablen als **ohne Wert**.

Definiert wird der Makro `NULL` in den Standard-Headerdateien

- `stddef.h`,
- `stdio.h`,
- `string.h`.

Trotz der Bezeichnung kann man sich nicht darauf verlassen, dass der Nullzeiger den Wert 0 hat und sollte immer `NULL` verwenden. So wird auch sofort klar, dass es sich um einen Zeiger handelt.

Parameter des Hauptprogramms

Jedes C-Programm besitzt mit der Funktion `main` ein Hauptprogramm.

Das Hauptprogramm wird beim Start des Programms vom Betriebssystem aufgerufen und erhält beim Aufruf zwei Argumente, die bislang ignoriert wurden.

```
int main (void)
```

Der Prototyp für `main` sieht tatsächlich anders aus.

```
int main (int argc, char *argv[]);
```

Übergeben bekommt `main` den bereits zerlegte Inhalt der Aufrufzeile des Programms.

- `argc` ist die Anzahl der Einträge in der Aufrufzeile (Leerzeichen o.Ä. werden als Trennzeichen interpretiert).
- `argv` ist ein Vektor von Zeigern auf die einzelnen Einträge (Strings).

Übergabe

```
int main (int argc, char *argv[]);
```

Der String, auf den `argv[0]` zeigt, ist der Name des Programms oder ein leerer String, wenn das Betriebssystem den Namen des Programms nicht liefert.

Die nächsten `argc - 1` Komponenten zeigen auf die weiteren Einträge der Aufrufzeile, in der Reihenfolge, in der sie in der Aufrufzeile angegeben sind.

Dem letzten *echten* Zeiger folgt noch eine Komponente mit dem Nullzeiger (`argv[argc]`), so dass man sich wahlweise an diesem Nullzeiger oder an dem Wert von `argc` orientieren kann.

Nicht übergeben werden dabei alle Einträge der Aufrufzeile, die sich auf die Umleitung der Ein- und Ausgabe oder auf Filter beziehen, da dieses vom Betriebssystem bearbeitet werden und für das Programm transparent sind.

Parameter

Was ein Programm mit seinen Parametern anfängt, ist ihm selbst überlassen. Es kann sie ignorieren, wie wir es bislang getan haben, oder aber auch darauf reagieren.

Beispiel

Ein Programm soll die Einträge seiner Aufrufzeile auflisten.

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[]) {
4      for (int i = 0; i < argc; i++)
5          printf("%s\n", argv[i]);
6      // OR
7      for(int i = 0; argv[i] != NULL; i++)
8          printf("%s\n", argv[i]);
9      return 0;
10 }
```

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

const links

Das Zusammenspiel zwischen Zeigervariablen und dem Schlüsselwort `const` ist komplex.

Bei der Deklaration von Zeigervariablen kann das Schlüsselwort `const` an verschiedenen Stellen stehen.

Steht `const` **links** von `*`, dann sind die von der Zeigervariablen referenzierten Werte konstant. Man kann der Zeigervariablen jedoch jederzeit einen neuen Zeiger (Wert) zuweisen.

An welcher Stelle `const` links von `*` steht, hat keine Bedeutung.

```
int i, j;
const int *p = &i;    // equals: int const *p = &i;

*p = 2;                // ERROR: reference value constant
p = &j;                // ok
```


const rechts

Rechts von * bewirkt **const**, dass die Zeigervariable selbst konstant ist.

D.h. einmal initialisiert kann man ihr keinen neuen Zeiger (Wert) zuweisen. Der von ihr referenzierte Wert kann jedoch geändert werden.

```
int i, j;
int * const p = &i;

*p = 3;    // ok
p = &j;    // ERROR: pointer constant
```

Eine Kombination aus beiden Fällen ist ebenfalls möglich. Es gelten dann entsprechend beide Einschränkungen für die Zeigervariable.

```
int i, j;
int const * const p = &i;

*p = 4;    // ERROR: reference value constant
p = &j;    // ERROR: pointer constant
```

Bedeutung merken durch lesen von rechts nach links liest. *p* ist ein (**konstanter**) Zeiger auf (**konstante**) *int*-Werte.

Verloren gehen des `const`-Attributs

Beim Umgang mit Zeigern und `const` ist besondere Aufmerksamkeit erforderlich.

Durch Zuweisungen kann das `const`-Attribut versehentlich verloren gehen.

Der Standard sieht das nicht als Fehler an und der Compiler produzieren entsprechend höchstens eine Warnung.

```
const int k = 42;
const int *p = &k;
int *q = &k;      // WARNING: const attribute is stripped of
*q = 43;          // value of k is 43

q = p;            // WARNING: ...
```

Funktionsparameter

Das Schlüsselwort `const` kann auch für Funktionsargumente mit der gleichen Bedeutung wie bei Variablendeklarationen verwendet werden.

In der Praxis wird man es dort sogar wesentlich häufiger antreffen.

Zeiger als Parameter von Funktionen werden in C verwendet, um *call by reference* zu simulieren.

Das Zeigerargument (der Zeiger) selbst wird kopiert, durch Dereferenzierung erhält man aber direkten Zugriff auf die eigentlichen Daten und kann diese ändern.

Mit einem Zeigerargument soll meist nur das unter Umständen aufwändige automatische Kopieren der Argumente umgangen werden, da sich bei großen Datenstrukturen die Parameterübergabe negativ auf die Laufzeit auswirken könnte.

Das Schlüsselwort `const` im Prototyp einer Funktion ist dann ein wertvoller Hinweis, dass die Daten nicht verändert werden.

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

Grundlegende Zeigerarithmetik (1/2)

Eine Eigenart von C ist, dass man mit Zeigern – in gewissen Grenzen – **rechnen** kann.

Der Ausgangspunkt dabei sind Felder.

Beispiel

```
int v[10];
```

```
v[7] = 9;
```

Es wird auf das Element zugegriffen wird, das den Abstand 7 vom Anfang des Feldes hat.

Anders formuliert. Der Zeiger auf den Anfang des Feldes, repräsentiert durch den Namen `v`, wird um 7 erhöht und der resultierende Wert für den Zugriff verwendet.

Das kann man auch wie folgt schreiben, mit gleicher Wirkung.

```
*(v + 7) = 9;
```

Die Klammern sind dabei **unbedingt** nötig, da der Dereferenzierungsoperator (wie alle unären Operatoren) eine höhere Priorität besitzt als die binären arithmetischen Operatoren.

Grundlegende Zeigerarithmetik (2/2)

Möglich ist auch Folgendes.

```
int v[10], *p;  
  
p = v + 7;  
*p = 9;           // equals v[7] = 9
```

Damit ist man bei *echter* Zeigerarithmetik angelangt.

Auch Operatoren mit Nebeneffekten und zusammengesetzte Operatoren sind erlaubt.

```
p++;  
p += 3;
```

Formulierungen von Feldzugriffen

Mit Zeigerarithmetik kommt man zu (formal) ganz anderen Formulierungen beim Zugriff auf Felder.

Beispiel

Funktion zur Berechnung der Summe der Komponenten eines Vektors.

```
float vector_sum(const float v[], int length) {  
    float sum = 0;  
    while (--length >= 0)  
        sum += v[length];  
    return sum;  
}
```

Formulierungen mit Zeigerarithmetik.

```
float vector_sum(const float *v, int length) {  
    float sum = 0;  
    while (length--)  
        sum += *v++;  
    return sum;  
}
```

Beispiel

```
float vector_sum(const float *v, int length) {  
    float sum = 0;  
    while (length--)  
        sum += *v++;  
    return sum;  
}
```

Anmerkungen.

- Die Parameterdefinitionen `const float v[]` und `const float *v` sind gleichwertig.
- Beim Aufruf wird der Wert des ersten Arguments in die lokale Variable `v` der Funktion **kopiert**, so dass die Änderung von `v` durch `v++` auch dann zulässig ist, wenn das Argument eine Konstante ist.
- Beim Ausdruck `*v++` wird ausgenutzt, dass die unären Operatoren alle auf derselbe Hierarchiestufe stehen und dass sie ggf. von rechts nach links abgearbeitet werden.

Der Ausdruck ist also gleichwertig mit `*(v++)`.

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

Operationen mit Zeigern

Welche Operationen sind für Zeiger zulässig?

- Zeigern können Werte mit dem **Zuweisungsoperator** zugewiesen werden.
- Ganze Zahlen können auf Zeiger addiert und von ihnen subtrahiert werden.

Man kann dabei jeden Operatoren verwenden, der auch für Addition und Subtraktion von ganzen Zahlen zulässig ist.

```
char *p;  
int i;
```

```
...
```

```
p++  
p + i  
p--  
p - i
```

Was logisch sinnvoll ist, steht auf einem anderen Blatt.

Vergleich

- Zeiger können verglichen werden, alle 6 Vergleichsoperatoren sind zulässig. Der Ablauf entspricht der Bildung der Differenz von zwei Zeigern. Es wird unterstellt, dass beide Zeiger auf Komponenten desselben Vektors zeigen. Der Vergleich liefert dasselbe Resultat wie ein Vergleich der Indizes.

Beispiel

```
float vector_sum(const float *v, int length) {  
    float sum = 0, *p = v + length;  
    while (v < p)  
        sum += *v++;  
    return sum;  
}
```

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

Zeiger als Parameter (II)

Bei Funktionen ist es letztlich gleichgültig, ob einen Parameter als Vektor oder als Zeiger deklarieren wird.

Zeiger sind typischer für C und erlauben vielfach extrem kompakte Formulierungen von Funktionen.

Beispiel. Kopieren eines String.

Felder

```
void strcpy1(char s[], const char t[]) {  
    int i = 0;  
    while (s[i] = t[i])  
        i++;  
}
```

Zeiger

```
void strcpy2(char *s, const char *t) {  
    while (*s++ = *t++)  
        ;  
}
```

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

Zeigervektoren

Erinnerung, Anfangswerte.

```
const char str3[] = "hello";  
const char *const str5 = "hello";
```

- **str3** ist formal eine Konstante.

Sie besitzt keinen Speicherplatz, sondern zeigt selbst auf den Anfang des konstanten String.

- **str5** ist eine Variable.

Sie besitzt einen Speicherplatz, so dass erst ihre Dereferenzierung den Zeiger auf den Anfang des konstanten Strings liefert.

Beispiel (1/3)

Zeigervariablen mit konstanten Werten sind sinnvoll.

Beispiel

Auf Zahlungsanweisungen und anderen Belegen müssen Beträge in Buchstaben wiederholt werden. Man behilft sich gelegentlich damit, die Ziffern einzeln in Buchstaben umzusetzen und irgendein Trennzeichen dazwischen zu setzen, so dass etwa 138 die Zeichenfolge folgendes ergibt.

```
one = three = eight
```

Die Buchstabenfolgen wird man in einem Feld speichern.

```
const char digits[][6] =  
{  
    "zero", "one", "two", "three", "four",  
    "five", "six", "seven", "eight", "nine"  
};
```

Dabei *verschwendet* man Speicherplatz. Jede Zeile der Matrix kann 6 Zeichen aufnehmen; benötigt werden diese 6 Zeichen aber nicht für alle Strings.

Eine 60 Zeichen große Matrix wird bereitgestellt, aber nur 50 Byte werden benötigt.

Beispiel (2/3)

Bei einer Folge von 100 Strings, von denen einer 99 Zeichen, die übrigen dagegen nur jeweils 4 Zeichen lang sind, macht es einen gewaltigen Unterschied, ob man eine (100×100) -Matrix definiert und damit 10000 Bytes belegen, oder nur die benötigten 595 Bytes für die Strings verwendet.

Man kann das Problem lösen, indem man einen Vektor von Zeigern auf Strings definiert.

```
const char *const digits[] =  
{  
    "zero", "one", "two", "three", "four",  
    "five", "six", "seven", "eight", "nine"  
};
```

Jede Komponente des Vektors ist ein Zeiger auf einen String; die Strings selbst speichert der Compiler irgendwo, mit genau der Länge, die sie einschließlich des abschließenden Null-Zeichens haben.

Im konkreten Beispiel ist diese Lösung aufwendiger als die erste Formulierung. Bei den Strings spart man zwar 10 Zeichen, aber dafür definiert man zusätzlich einen Zeigervektor mit 10 Zeigern, der garantiert mehr Speicherplatz als 10 Zeichen benötigt.

Beispiel (3/3)

Nach der Vorüberlegungen zu den Daten erweist sich die Realisierung als einfach.

Eine rekursive Funktion erledigt die eigentliche Arbeit.

Sie wird in einen nichtrekursiven Rahmen eingebettet, weil der Trennstring nur zwischen je zwei Ziffern geschrieben werden darf, der dafür erforderliche zusätzliche Parameter für den Benutzer der Funktion aber *versteckt* werden soll.

```
1  #include <stdio.h>
2  #include <stdbool.h>                                // _Bool, true/false
3
4  //...
5  void pretty_print_recursive(int value, _Bool separator);
6
7  //-----
8  void pretty_print(int value) {
9      pretty_print_recursive(value, false);
10 }
11
12 void pretty_print_recursive(int value, _Bool separator) {
13     if (value >= 10)
14         pretty_print_recursive(value/10, true);
15     printf ("%s", digits[value%10]);
16     if (separator)
17         printf ("□=□");
18 }
```

Zeiger

Parameter des Hauptprogramms

Zeiger und const

Zeigerarithmetik

Operationen mit Zeigern

Zeiger als Parameter (II)

Zeigervektoren

Zeiger auf Zeiger

Zeiger auf Zeiger

Der im vorherigen Beispiel wurde ein Vektors von Zeigern verwendet.

```
const char *const digits[] =  
{  
    "zero", "one", "two", "three", "four",  
    "five", "six", "seven", "eight", "nine"  
};
```

Der Name des Vektors ist der Zeiger auf den Anfang einer Folge von Variablen, die ihrerseits Zeiger auf Strings sind.

Zeiger auf Zeiger kann man explizit definieren.

```
char **table;
```

`table` ist ein Zeiger auf eine Variable mit dem Typ `char *`, also Zeiger auf eine Variable mit einem Zeigertyp.

Beispiel

Beispiel

Ein Programm soll die Einträge seiner Aufrufzeile auflisten.

Realisierung mit Zeigerarithmetik.

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[]) {
4      for (char **p = argv; *p != NULL; p++)
5          printf("%s\n", *p);
6      return 0;
7  }
```

Allgemein

Mehrdimensionale Felder entsprechen Zeigern auf Zeiger

Beispiel

```
int m[2][3];
```

- `m` ist Zeiger auf den Vektor der Zeilenvektoren.
- `m[i]` ist der Zeiger auf das erste Element in der $(i + 1)$ -ten Zeile der Matrix, einen *zulässigen* Wert von `i` vorausgesetzt.

Damit hat man mehrere Möglichkeiten auf ein bestimmtes Element zuzugreifen (zulässige Werte der Indizes vorausgesetzt).

```
m[i][j]  
(*(m + i))[j]  
*(m[i] + j)  
*(*(m + i) + j)
```